LOS NÁUFRAGOS · PROGRAMACIÓN EN AMBIENTE WEB 2 · SEXTO SEMESTRE

Planificador de procesos

Se debe construir un simulador que recibe una lista de procesos, les aplica los algoritmos de planificación de CPU y muestra el diagrama de Gantt de cada uno junto a sus métricas. El sistema permite responder con datos —y no de memoria— la pregunta central del dominio: para esta carga, ¿cuál algoritmo conviene?

Equipo Los Náufragos: Kamila Marín Cossio · Juan Fernando Muñoz Silva

---

El problema

El sistema no automatiza un ejercicio inventado para practicar Angular: automatiza un procedimiento que hoy se resuelve a mano, y que a mano se hace mal.

Hoy, comparar algoritmos de planificación se hace en el tablero: se dibuja la línea de tiempo con marcador, se van marcando las rebanadas de CPU, y luego se suman los tiempos de espera de cada proceso para obtener el promedio. Con cuatro procesos y FIFO el procedimiento se sostiene. Con siete procesos, Round Robin de quantum 3 y llegadas escalonadas, el diagrama ocupa media pizarra y basta con equivocarse en un instante para que todas las métricas que vienen después queden mal —y el error no se nota, porque no hay contra qué compararlo.

El problema de fondo es peor que la aritmética. Comparar cuatro algoritmos sobre la misma carga significa dibujar cuatro diagramas completos, y casi nunca hay tiempo para eso. Entonces la pregunta importante —*¿cuál conviene para esta carga?*— termina contestada con una regla memorizada: "SJF minimiza el tiempo de espera promedio". Es cierta, pero solo bajo supuestos que rara vez se enuncian, y no dice nada sobre el tiempo de respuesta, ni sobre qué pasa cuando llega un proceso largo primero, ni sobre cómo cambia todo si el quantum es 2 en vez de 5.

El sistema convierte esa comparación en un procedimiento de treinta segundos: se captura la carga una vez, se ejecutan los cuatro algoritmos sobre ella, y se presentan los cuatro diagramas y las tres métricas lado a lado. La conclusión deja de ser una cita del libro y pasa a ser la lectura de una tabla.

Alcance

Delimitar lo que no se pide es tan importante como listar lo que sí. El alcance está acotado de forma deliberada y el sistema se evalúa contra ese alcance.

QUÉ ENTRA

- Definir procesos con **`llegada`**, **`ráfaga`** y **`prioridad`**
- Guardar conjuntos de procesos y reutilizarlos
- **FIFO** (primero en llegar, primero servido)
- **SJF sin desalojo** y **SJF con desalojo** (SRTF)
- **Round Robin** con quantum configurable
- **Prioridades**, con la regla de desempate documentada
- Diagrama de Gantt con la CPU ociosa marcada
- Tiempo de espera, de retorno y de respuesta, por proceso y promedio
- Comparación de los cuatro algoritmos sobre el mismo conjunto
- Biblioteca nativa **`libplanificador.so`** invocada con JNA

QUÉ NO ENTRA

- Gestión de memoria — ampliación opcional, ver el cierre del documento
- Planificación en varios núcleos o varias CPU
- Planificación de tiempo real con vencimientos
- Colas multinivel con retroalimentación — *salvo* como demostración del escenario RQ-01
- Estados completos del proceso: bloqueo por E/S, suspensión, intercambio
- Envejecimiento de prioridades para evitar inanición
- Cuentas de usuario, roles y permisos
- Ejecución real de procesos del sistema operativo: esto es una simulación

> **Sobre la lista de la derecha.** Ninguno de esos elementos queda fuera por ser difícil: cada uno es un proyecto en sí mismo y ninguno aporta un aprendizaje que el núcleo no aporte ya. Una vez terminado el núcleo, cualquier ampliación se acuerda con el docente; no se decide de forma unilateral a mitad de camino.

Qué evalúa cada curso

El sistema es uno solo, pero se evalúa desde dos cursos con entregables distintos. Saber cuál es cuál evita la discusión de última hora sobre quién responde por qué.

PROGRAMACIÓN EN AMBIENTE WEB 2

La aplicación. El frontend en Angular, los microservicios Spring Boot, el modelo de datos, **la autenticación y la autorización**, los tres escenarios de calidad con su medición, y la bitácora de decisiones.

ARQUITECTURA DE COMPUTADORES — LOS NÁUFRAGOS

El componente nativo y el despliegue. La biblioteca `libplanificador.so` en C, la interfaz JNA, el servicio que la expone, la medición de C contra Java, y el **apoyo al despliegue de la aplicación a producción**.

> **La autenticación y la autorización no se exigen en Arquitectura de Computadores.** Ese bloque pertenece por completo a Web 2. El servicio que expone la biblioteca nativa queda protegido igual que los demás, pero configurarlo y demostrarlo es responsabilidad del frente de Web 2, no del frente nativo.

QUÉ SIGNIFICA "APOYAR EL DESPLIEGUE A PRODUCCIÓN"

No es una frase de cortesía: son entregables concretos, y salen naturalmente de quien construye el componente nativo, porque es quien tiene que incorporar una biblioteca compilada a una imagen.

> 01
> **La imagen del servicio nativo**
> Un `Dockerfile` multietapa: una etapa compila `libplanificador.so` con `gcc`, la siguiente copia solo la biblioteca y el JAR. La imagen final no lleva compilador.

> 02
> **El manifiesto de despliegue completo**
> El YAML de Kubernetes o el archivo de stack de Docker Swarm de *todo* el sistema, con los límites de CPU y memoria declarados explícitamente, no solo los del servicio nativo.

> 03
> **El arranque desde cero comprobado**
> Un solo comando levanta el sistema en una máquina limpia, sin pasos manuales y sin que nadie tenga que conocer el orden. Se comprueba en una máquina donde el proyecto nunca se había levantado.

> 04
>
> **El costo del despliegue, medido**
>
> Tamaño de la imagen y tiempo de arranque del contenedor **antes y después** de incorporar la biblioteca nativa. Bajar un cálculo a C cuesta algo en el despliegue, y ese costo se reporta con números.

EL CONTRATO ENTRE LOS DOS FRENTES

Lo que une los dos entregables es una **interfaz**, no una reunión. Se acuerda en la primera semana, se congela y queda escrita en la bitácora de ambos frentes:

- La **interfaz JNA** exacta: nombres de función, tipos de parámetro y de retorno, y quién libera la memoria que se reserva.
- Un conjunto de **casos de referencia** con su valor esperado, escritos antes de implementar, que sirven de oráculo a los dos lados.
- Cómo se reporta un error desde C: código de retorno, no excepción, porque C no tiene.

> **La aplicación debe funcionar sin la biblioteca nativa.** Todos los proyectos exigen ya una implementación en Java de la misma función, como línea base para medir si bajar a C valió la pena. Esa línea base es también el *respaldo*: `planificador-service` escoge entre la ruta nativa y la de Java por configuración, de modo que si el componente nativo no está listo la aplicación sigue en pie y la evaluación de Web 2 no queda a merced de otro frente. No es trabajo extra — ya estaba pedido para la medición—, pero sí una decisión de diseño que debe quedar escrita.

Requisitos funcionales

Diez requisitos, tres de ellos opcionales. Cada uno trae un criterio de aceptación: la forma concreta de saber que quedó hecho. Un requisito que no se puede verificar no se puede calificar; un criterio que resulte ambiguo al implemen-

tarlo debe redactarse de nuevo y anotarse en la bitácora.

RF 01

Registrar un proceso

El usuario captura un proceso con su identificador, tiempo de llegada, duración de ráfaga y prioridad. Los identificadores no se repiten dentro de un mismo conjunto.

> **Criterio:** se crea un conjunto de cinco procesos, se reinician los contenedores, y el conjunto sigue ahí con los mismos valores.

RF 02

Guardar y recuperar conjuntos de prueba

Un conjunto es una carga con nombre —"parcial 1", "todos llegan en cero"— que se puede volver a cargar sin recapturar nada.

> **Criterio:** desde la pantalla inicial se carga un conjunto guardado y queda listo para ejecutar sin escribir un solo número.

RF 03

Ejecutar un algoritmo sobre un conjunto

El usuario elige un algoritmo de la lista y obtiene la línea de tiempo resultante y sus métricas.

> **Criterio:** FIFO sobre el conjunto de control resuelto a mano produce exactamente los mismos instantes de inicio y fin de cada rebanada.

RF 04

Configurar el quantum de Round Robin

El quantum es un parámetro de la corrida, no una constante del código.

> **Criterio:** la misma carga con quantum 2 y con quantum 5 devuelve distinto número de rebanadas y distintos promedios, sin recompilar ni redesplegar nada.

RF 05

Ver el diagrama de Gantt

La línea de tiempo con una barra por rebanada, el eje de tiempo con sus marcas, y la CPU ociosa visualmente distinta del trabajo.

> **Criterio:** con ocho procesos, el diagrama se lee completo en una pantalla de 1366 px sin desplazamiento horizontal, y cada barra deja ver a qué proceso pertenece y entre qué instantes va.

RF 06

Calcular las tres métricas

Tiempo de espera, tiempo de retorno y tiempo de respuesta, para cada proceso y promediados. El promedio lo calcula el backend, no el navegador.

> **Criterio:** los valores por proceso y los tres promedios coinciden con la tabla calculada a mano sobre el conjunto de control.

RF 07

Comparar los cuatro algoritmos

Una sola acción ejecuta todos los algoritmos disponibles sobre el mismo conjunto y presenta los resultados juntos.

> **Criterio:** en una pantalla se ven los cuatro promedios de espera, retorno y respuesta, y se identifica sin ambigüedad cuál algoritmo gana en cada métrica.

RF 08

Rechazar entradas inválidas con un mensaje útil

Ráfaga mayor que cero, llegada no negativa, quantum mayor o igual a uno. El mensaje dice qué campo está mal y qué valor se esperaba.

> **Criterio:** capturar una ráfaga de 0 produce un mensaje entendible en la interfaz; no una pantalla en blanco, ni un error 500, ni un cierre del contenedor.

RF 09

Ver la traza de decisión `opcional`

En cada instante en que el planificador escoge, mostrar qué procesos estaban listos y por qué se eligió ese. Es lo que convierte el simulador en material de estudio.

> **Criterio:** para una corrida de SRTF se puede desplegar, instante por instante, la cola de listos y el criterio aplicado.

RF 10

Exportar la comparación `opcional`

Descargar la tabla comparativa en CSV o el diagrama en imagen, para incorporarlo a un informe.

> **Criterio:** el archivo descargado abre en una hoja de cálculo con los mismos números que muestra la pantalla.

RF 11

Sembrar conjuntos de ejemplo al arrancar `opcional`

El sistema arranca con dos o tres cargas clásicas ya cargadas, para que quien lo abre por primera vez tenga algo que ejecutar de inmediato.

> **Criterio:** en un despliegue limpio, sin capturar nada, se puede ejecutar una comparación completa.

Requisitos de calidad

Esta es la parte que define la arquitectura. Los requisitos funcionales dicen qué hace el sistema; estos dicen **cómo tiene que estar construido**, y deben demostrarse con una medición en la sustentación. El primero es el atributo asignado y no es negociable. Los otros dos vienen ya redactados por ser los que este proyecto exige.

RQ-01 · ATRIBUTO ASIGNADO

Modificabilidad — agregar un quinto algoritmo

Un simulador de planificación es, por naturaleza, una colección de algoritmos que se va a ampliar. Si agregar el quinto cuesta lo mismo que escribir el primero, el diseño falló. Este escenario mide exactamente eso, y debe demostrarse en vivo.

FUENTE	Un integrante del equipo, actuando como desarrollador del sistema.

ESTÍMULO	Llega el requerimiento de agregar un quinto algoritmo de planificación que no existía cuando se diseñó el sistema —por ejemplo, colas multinivel con retroalimentación—.
ARTEFACTO	El sistema completo: `libplanificador.so`, `planificador-service`, el contrato REST y el frontend Angular.
ENTORNO	Sistema ya terminado y desplegado, en tiempo de desarrollo. Se hace en frío, sin nada preparado de antemano y sin ramas escritas por adelantado.
RESPUESTA	El algoritmo queda disponible para el usuario modificando **únicamente el código C y reconstruyendo la imagen** de `planificador-service`. No se toca ninguna clase de dominio en Java, ningún endpoint, ningún archivo del frontend. El algoritmo nuevo aparece solo en el selector de Angular, porque la lista de algoritmos la entrega la biblioteca y viaja por la API.
MEDIDA	**Módulos tocados ≤ 2** (el archivo `.c` y su cabecera). **0 archivos de Angular modificados. 0 cambios en el contrato OpenAPI.** Tiempo total desde el primer teclazo hasta verlo corriendo en el navegador: **≤ 30 minutos**, cronometrados delante del docente.

Cómo se mide: se hace un commit antes de empezar y otro al terminar, y se presenta el `git diff --stat` entre los dos. Ese diff es toda la evidencia: si aparece un `.ts` en la lista, el escenario no se cumplió. Debe añadirse el `diff` del archivo OpenAPI para probar que el contrato quedó intacto.

> **Esto condiciona decisiones de la semana 1, no de la última.** Si el algoritmo viaja en la API como un texto libre validado contra la lista que reporta la biblioteca, el escenario se cumple solo. Si se escribe como un `enum` en Java y como un arreglo fijo en Angular, agregar uno obliga a tocar tres módulos y la medida se pierde. Igual con la respuesta: si el JSON de salida es genérico —rebanadas más métricas— sirve para cualquier algoritmo; si tiene campos con nombre de algoritmo dentro, no sirve para el siguiente. **Ese es el aprendizaje del proyecto**, y por eso el quinto algoritmo no va en el núcleo: queda reservado como la prueba del escenario.

RQ-02 · ESCENARIO PROPIO

Testabilidad — las métricas tienen que poder comprobarse

Que los números estén bien es funcional; lo arquitectural es poder demostrarlo a demanda, y eso es lo que este escenario exige. Todo el valor del sistema depende de que se pueda confiar en los números. Un simulador que dibuja un Gantt impecable con un

tiempo de espera mal calculado es peor que no tener nada, porque nadie lo va a revisar a mano.

FUENTE	El docente durante la sustentación.
ESTÍMULO	Entrega un conjunto de seis procesos desconocido para el equipo, ya resuelto a mano, y solicita ejecutar los cuatro algoritmos.
ARTEFACTO	Los algoritmos en `libplanificador.so` y el cálculo de métricas que hace `planificador-service`.
ENTORNO	Operación normal, con datos desconocidos, sin ajustar nada entre la captura y el resultado.
RESPUESTA	El diagrama y las tres métricas coinciden con el cálculo de control. Los desempates —dos procesos que llegan en el mismo instante, dos ráfagas iguales, dos prioridades iguales— siguen una regla **documentada y visible en la interfaz**, no la que salga del orden del arreglo.
MEDIDA	**0 diferencias** en los instantes de inicio y fin de cada rebanada. **0 diferencias** en los promedios a dos decimales. Verificado sobre **6 conjuntos de control**, uno por caso borde: llegadas simultáneas, empate de ráfaga, hueco de CPU ociosa, quantum mayor que la ráfaga más larga, un solo proceso, prioridades repetidas.

Cómo se mide: una suite de pruebas JUnit con los seis conjuntos y sus resultados esperados escritos a mano —no generados por el propio sistema—, que corre en cada construcción. Un caso borde sin prueba es un caso borde que todavía no existe.

RQ-03 · ESCENARIO PROPIO

Usabilidad — el Gantt es lo que la gente mira

El usuario no lee el JSON. Lo único que ve es el diagrama y la tabla comparativa, y si su lectura requiere la explicación de un integrante del equipo, el sistema no sirve para lo que se hizo. Este escenario se mide con personas, no con un cronómetro de servidor.

FUENTE	Una persona ajena al equipo, conocedora del dominio de planificación de procesos, que nunca ha visto el sistema.
ESTÍMULO	Quiere saber cuál algoritmo da el menor tiempo de espera promedio para una carga de cinco procesos que trae anotada en el cuaderno.

ARTEFACTO	El frontend Angular: la captura de procesos, el diagrama de Gantt y la tabla comparativa.
ENTORNO	Primera vez que lo usa, en un portátil, sin instrucciones previas y sin intervención del equipo.
RESPUESTA	Captura los cinco procesos, ejecuta la comparación y responde su pregunta sin ayuda. El diagrama se entiende sin explicación externa: eje de tiempo rotulado, cada barra identificada con su proceso, y la CPU ociosa distinguible de un vistazo.
MEDIDA	**≤ 12 interacciones** (clics o pulsaciones) desde la pantalla inicial hasta la comparación. **≤ 5 minutos. 0 preguntas** al observador. Repetido con **3 personas distintas** ajenas al equipo.

Cómo se mide: tres sesiones cronometradas, contando interacciones y anotando textualmente cada pregunta que la persona hace en voz alta. Las preguntas son el resultado más valioso: se registran en la bitácora aunque no alcancen a corregirse.

ESCENARIOS ADICIONALES

Todo escenario que el equipo agregue se escribe con estas mismas seis partes y con una medida numérica. La plantilla se copia tal cual:

FUENTE	¿Quién o qué provoca el estímulo? Una persona, otro servicio, un reloj, una falla.
ESTÍMULO	¿Qué ocurre exactamente? Una acción concreta, no una situación general.
ARTEFACTO	¿Sobre qué parte del sistema cae? Se nombra: un servicio, la biblioteca, el frontend.
ENTORNO	¿En qué estado está el sistema? Operación normal, carga alta, un servicio caído, tiempo de desarrollo.
RESPUESTA	¿Qué hace el sistema? Observable desde afuera.
MEDIDA	¿Con qué número se comprueba? Milisegundos, módulos tocados, MB, interacciones, diferencias contra un control.

> **Los nombres de atributo son cerrados**, y son los de Len Bass, *Software Architecture in Practice*: **Disponibilidad**, **Facilidad de despliegue**, **Eficiencia energética**, **Integrabilidad**, **Modificabilidad**, **Desempeño**, **Seguridad**, **Seguridad física**, **Testabilidad** y **Usabilidad**. No se aceptan nombres inventados ni sinónimos sueltos —"robustez", "precisión", "eficiencia" a secas, "calidad"—: un escenario mal nombrado es un escenario que no se puede discutir contra el de otro proyecto ni contrastar con la literatura.
>
> **Tres nombres de uso frecuente quedan fuera, y conviene saber por qué.** *Escalabilidad* no es un atributo aparte en Bass: el comportamiento cuando la carga o los datos crecen se expresa como un escenario de **Desempeño**, con el volumen puesto en el estímulo y el crecimiento del tiempo en la medida. *Eficiencia de recursos* tampoco: el consumo de memoria, CPU o disco va en el entorno y en la medida de un escenario de **Desempeño**. Y que el resultado sea el correcto es un asunto **funcional**, no un atributo de calidad —Bass los separa a propósito—: lo arquitectural es con cuánta facilidad el sistema puede demostrar que no se equivoca, y eso se llama **Testabilidad**.

Autenticación y autorización

Ningún endpoint del sistema queda abierto. La identidad de quien hace cada petición se establece con **OAuth 2.0 y OpenID Connect**, y el proveedor de identidad es **Keycloak**, desplegado como un contenedor más del mismo manifiesto. Estos requisitos son obligatorios y se evalúan igual que los funcionales: con una comprobación, no con una afirmación.

> **Este bloque corresponde únicamente a Programación en Ambiente Web 2.** Arquitectura de Computadores no lo evalúa.

POR QUÉ SE DELEGA LA IDENTIDAD Y NO SE PROGRAMA

Escribir un inicio de sesión propio —una tabla de usuarios, una contraseña con hash, una cookie de sesión— es más corto de programar que integrar un proveedor externo, y por eso es la salida habitual. También es la que no sobrevive al primer microservicio adicional: la sesión vive en un proceso y el segundo servicio no la conoce, de modo que o se comparte el almacenamiento de sesiones —y los servicios dejan de ser independientes— o cada uno vuelve a pedir credenciales.

Un token firmado resuelve exactamente eso. Keycloak emite un **JWT** firmado con su llave privada; cada servicio verifica la firma contra la llave pública publicada por el realm y obtiene la identidad y los roles *sin consultar a nadie* y sin guardar estado. Esa es la razón arquitectural, y es la que debe quedar escrita en la bitácora: la decisión no es "usar Keycloak porque se pidió", sino que la validación de identidad debe poder ocurrir en cada servicio de forma independiente.

REQUISITOS

RA-01 **Keycloak desplegado como parte del sistema**

Un contenedor de Keycloak con un realm propio, `planificador-realm`, levantado por el mismo manifiesto que el resto. El realm define los roles, los clientes y al menos un usuario de prueba por rol.

Criterio El realm se importa desde un archivo versionado en el repositorio. Al levantar el sistema desde cero en una máquina limpia, los roles y los usuarios de prueba ya existen, sin configuración manual en la consola de Keycloak.

RA-02 **Inicio de sesión desde Angular con Authorization Code y PKCE**

El frontend se registra como cliente **público** y usa el flujo de código de autorización con PKCE. No hay *client secret* en el navegador, y la pantalla de credenciales es la de Keycloak: la aplicación nunca ve la contraseña.

Criterio Al abrir una ruta protegida sin sesión, el navegador es redirigido a Keycloak; tras autenticarse regresa a la ruta solicitada. En el código fuente del frontend no aparece ninguna credencial ni *secret*, y en las herramientas del navegador se observa el parámetro `code_challenge` en la petición de autorización.

RA-03 **El token viaja en cada petición, puesto por un interceptor**

El *access token* se adjunta como `Authorization: Bearer …` mediante un interceptor funcional de Angular, no repitiendo la cabecera en cada llamada. El frontend **no decide** si el usuario tiene permiso: solo adjunta el token y reacciona a la respuesta.

Criterio Todas las peticiones a la API salen con la cabecera. Ocultar un botón según el rol es aceptable como comodidad visual, pero eliminar esa ocultación desde el navegador y disparar la petición debe seguir produciendo un rechazo del backend.

RA-04 **Cada microservicio valida el token por su cuenta**

`planificador-service` y `proceso-service` se configuran como *resource server* de OAuth 2.0. Cada uno verifica la firma del JWT contra el JWKS del realm y rechaza el token vencido, con firma inválida, con emisor distinto o con destinatario que no le corresponde.

Criterio Una petición sin token responde **401**. Una petición con un token al que se le alteró un carácter de la firma responde **401**. Ambas deben comprobarse contra cada servicio por separado, no solo a través del frontend.

RA-05 **Autorización por rol en cada endpoint**

Los roles del realm son `docente` y `estudiante`, y determinan qué operaciones admite cada endpoint según la matriz de más abajo. La autorización se declara en el backend, sobre el endpoint, no en el frontend.

Criterio Un token válido con el rol equivocado responde **403**, no 401 y no 200. La diferencia importa: 401 significa "no se sabe quién es"; 403 significa "se sabe quién es y no le corresponde". Debe comprobarse en ambos sentidos: un `estudiante` intentando registrar un conjunto de referencia y un `estudiante` intentando eliminar un conjunto.

RA-06 **La identidad queda en los datos**

Cada corrida guarda qué usuario la ejecutó, con qué algoritmo y sobre qué conjunto de procesos. Se guarda el identificador estable del usuario que trae el token —el *claim* `sub`— y no solo el nombre visible, que puede cambiar.

Criterio Tras operar con dos usuarios distintos, la consulta a la base de datos muestra dos identificadores diferentes en los registros correspondientes, y ese dato proviene del token, no de un campo enviado por el frontend.

RA-07 **Expiración y cierre de sesión**

El *access token* tiene una vigencia corta declarada en el realm. Al vencerse, la aplicación lo renueva de forma transparente o conduce nuevamente al inicio de sesión. El cierre de sesión invalida la sesión en Keycloak, no solo borra el token del navegador.

Criterio Con la vigencia del token reducida a un minuto para la demostración, la aplicación sigue funcionando pasado ese minuto —porque renovó— o redirige al inicio de sesión, pero en ningún caso muestra un error sin explicación. Después de cerrar sesión, volver atrás en el navegador no devuelve acceso a la aplicación.

LOS DOS ROLES

DOCENTE	ESTUDIANTE
Administra los conjuntos de procesos de referencia sobre los que se corre el simulador.	Ejecuta simulaciones sobre los conjuntos disponibles y consulta sus resultados.

El administrador de Keycloak no es un rol de la aplicación: sirve para configurar el realm y no debe usarse para operar el sistema. Si hiciera falta un tercer rol de aplicación, se agrega y se justifica en la bitácora, pero los dos anteriores son el mínimo, porque son los que permiten demostrar un rechazo por rol.

MATRIZ DE AUTORIZACIÓN

Esta tabla es el contrato. Cada celda con **No** es una respuesta **403** que debe poder provocarse en la sustentación.

OPERACIÓN	DOCENTE	ESTUDIANTE	SIN AUTENTICAR
Ejecutar una simulación de planificación	**Sí**	**Sí**	No
Registrar un conjunto de procesos de referencia	**Sí**	**No**	No
Eliminar un conjunto de referencia	**Sí**	**No**	No
Consultar el diagrama de ejecución y las métricas	**Sí**	**Sí**	No
Consultar el historial propio de corridas	**Sí**	**Sí**	No

Hay dos decisiones que deben tomarse y quedar escritas.

1. Dónde se valida la autorización. Puede validarse en un solo punto de entrada, que concentra la regla y deja los servicios internos simples pero los vuelve inseguros si alguien alcanza la red interna; o puede validarse en cada servicio, que repite configuración pero deja a cada uno defendible por separado. Ninguna de las dos es la correcta: se evalúa la que quede justificada contra un escenario de calidad, con la alternativa descartada anotada.

2. Cómo se leen los roles del token. Keycloak coloca los roles del realm **anidados** dentro del *claim* `realm_access`, en la forma `realm_access.roles`. La configuración más citada de Spring Security para extraer autoridades busca el *claim* de forma **plana**, en el primer nivel del token, y por lo tanto no encuentra nada: el usuario queda autenticado pero sin autoridades, y todo responde 403 aunque el rol esté correctamente asignado en Keycloak. Las salidas son escribir un convertidor propio que descienda al *claim* anidado, o declarar en Keycloak un *mapper* que publique los roles en el primer nivel. La que se elija se anota, porque es el error que más tiempo consume en este punto.

CÓMO SE COMPRUEBA

La evidencia son cuatro respuestas HTTP, obtenidas contra el servicio y no contra el frontend. Se obtiene un token del realm pidiéndolo al endpoint de token de Keycloak con las credenciales de un usuario de prueba, y con ese token se ejecutan las cuatro llamadas: **sin cabecera de autorización** debe responder 401; **con un token cuya firma fue alterada**, 401; **con un token válido del rol que no corresponde**, 403; **con el token del rol correcto**, 200 con la respuesta esperada. Las cuatro se entregan como registro reproducible —comando ejecutado y respuesta obtenida—, y la que devuelve 403 debe mostrarse en los dos sentidos de la matriz. Un sistema donde todo responde 200 con cualquier token no cumple este bloque, aunque la pantalla de inicio de sesión funcione.

Restricciones

Estas condiciones no se negocian, y cada una tiene una razón. Conocer la razón permite discutir el resto del diseño con criterio.

FRONTEND

Angular con componentes standalone y signals. **El frontend no calcula nada del planificador**, ni siquiera un promedio: solo pide y dibuja. Es la condición para que RQ-01 se pueda cumplir.

BACKEND

Spring Boot. El sistema se compone de **dos microservicios**. La separación de responsabilidades entre ambos tiene que ser real y estar justificada.

CÓMPUTO NATIVO

Obligatorio. Los algoritmos van en C, compilados como `libplanificador.so` dentro de la imagen e invocados con **JNA**.

ORQUESTACIÓN

Kubernetes o **Docker Swarm**. Todo el sistema se levanta con un manifiesto y un comando. Nadie arranca servicios a mano en la sustentación, ni explica que "en mi máquina sí corre".

PERSISTENCIA

`proceso-service` tiene su propia base de datos. Si `planificador-service` necesita una o puede quedar sin estado, **lo define el equipo y lo justifica** en la bitácora.

HISTORIA DEL REPOSITORIO

Un repositorio con commits de **todos los integrantes**. Un historial donde solo aparece un autor es un dato de evaluación, no un descuido.

IDENTIDAD

Keycloak como proveedor de identidad, con **OAuth 2.0** y **OIDC**. Obligatorio. **Por qué:** un token firmado permite que cada microservicio valide por su cuenta quién llama, sin compartir sesión y sin consultar a los demás. Los requisitos están en la sección de autenticación y autorización.

Arquitectura propuesta

Este es un punto de partida razonable, no un dictado. Si el equipo lo cambia, la justificación va en la bitácora contra alguno de los escenarios de calidad.

SERVICIO	DE QUÉ RESPONDE	QUÉ GUARDA	CON QUIÉN HABLA
`proceso-service`	Procesos y conjuntos de prueba: crear, listar, validar. Es el dueño de los datos de entrada.	Conjuntos con sus procesos: llegada, ráfaga, prioridad.	Recibe del frontend. No conoce ningún algoritmo.
`planificador-service`	Ejecutar una planificación: carga la biblioteca nativa, le pasa los procesos, arma la respuesta con rebanadas y métricas. También publica la lista de algoritmos disponibles.	Opcional: el historial de corridas. Queda a criterio del equipo.	Llama a `proceso-service` para traer el conjunto, o lo recibe en el cuerpo de la petición. El equipo elige y justifica.

El corte está donde está por una razón: lo que cambia seguido (los algoritmos) queda concentrado en un lado, y lo que cambia poco (los datos de entrada) en el otro. Ese es el criterio que sostiene RQ-01. Una división del tipo "uno guarda todo y el otro es la interfaz" haría caer el escenario de modificabilidad.

Dos preguntas que el equipo debe responder, con su respuesta anotada: ¿se incorpora un `api-gateway`? Con dos servicios puede ser sobre-ingeniería, y consignar "no se incorporó porque con dos servicios el costo supera el beneficio" es una decisión de arquitectura perfectamente válida —siempre que esté escrita—. Y: ¿`planificador-service` le pide el conjunto a `proceso-service`, o el frontend se lo envía completo? La primera opción acopla los dos servicios; la segunda hace la petición más pesada. Las dos se defienden; lo que no se defiende es no haberlo considerado.

La biblioteca nativa

Los cuatro algoritmos se escriben en C, se compilan como biblioteca compartida y se invocan desde Spring Boot con JNA. En Java no queda ni un ciclo de planificación: un algoritmo duplicado en Java "por si acaso" produce dos fuentes de verdad y ninguna confiable.

```
planificador.c  →  gcc -shared -fPIC  →  libplanificador.so  →  imagen Dock
```

LO QUE VA EN C

Una sola llamada por corrida, que devuelve la línea de tiempo completa y las métricas ya agregadas. Nada de una llamada por rebanada: cruzar la frontera Java↔C tiene un costo, y pagarlo una vez por corrida en lugar de cien veces es precisamente la decisión de diseño que debe defenderse.

```
typedef struct { int pid; int llegada; int rafaga; int prioridad; } Proceso;
typedef struct { int pid; int inicio; int fin; }              Rebanada;   /* pid = -1
→ CPU ociosa */
typedef struct { int pid; int espera; int retorno; int respuesta; } Metrica;

typedef struct {
    Rebanada rebanadas[MAX_REBANADAS];  int n_rebanadas;
    Metrica  metricas[MAX_PROCESOS];    int n_metricas;
    double   espera_prom, retorno_prom, respuesta_prom;
} Resultado;

/* Devuelve 0 si todo salió bien, o un código de error negativo. */
int planificar(const char *algoritmo, const Proceso *procs, int n,
               int quantum, Resultado *salida);

/* Los identificadores disponibles, separados por coma: "fifo,sjf,srtf,rr,prioridad"
*/
int listar_algoritmos(char *buffer, int tam);
```

`listar_algoritmos` parece un detalle y es la pieza clave de RQ-01: es lo que permite que Angular no tenga escrito en ninguna parte cuáles algoritmos existen. Dentro del `.c`, el despacho se resuelve con una tabla que asocia el nombre a un apuntador a función; agregar un algoritmo se reduce entonces a escribir la función y añadir una fila a la tabla.

LO QUE VA EN JAVA

```
public interface LibPlanificador extends Library {
    LibPlanificador INSTANCE = Native.load("planificador", LibPlanificador.class);

    int planificar(String algoritmo, Proceso.ByReference procs, int n,
                   int quantum, Resultado.ByReference salida);

    int listar_algoritmos(byte[] buffer, int tam);
}

@Service
public class PlanificadorNativo {
    public ResultadoDTO ejecutar(String algoritmo, List<ProcesoDTO> procesos, int
quantum) {
        // ... mapear a la estructura, invocar, traducir el resultado a DTO
    }
}
```

El `@Service` es la **única** clase de todo el backend que sabe que existe una biblioteca en C. Todo lo demás habla con DTOs. Si la frontera se filtra hacia el controlador, cambiar la biblioteca deja de ser barato.

> LA MEDICIÓN QUE HAY QUE PRESENTAR
>
> Debe compararse la biblioteca en C contra una implementación de referencia en Java de los mismos cuatro algoritmos, sobre cargas de **50, 500, 5.000 y 50.000 procesos**, 30 repeticiones cada una, reportando la mediana. Se agregan el **tamaño de la imagen** y el **tiempo de arranque del contenedor** con y sin el `.so`.
>
> Es posible que con diez procesos el nativo *no* gane, porque el cruce JNA cuesta más que el algoritmo entero. Eso no constituye un fracaso del trabajo: es el resultado. Lo que se evalúa es identificar **dónde está el punto de cruce** y poder explicar qué gana y qué cuesta el sistema por tener ese componente ahí. Compilar la biblioteca sin poder justificar la decisión con un número no cumple el requisito.

Entregables

Lo que se recibe, y en qué forma. El sistema funcionando es la mitad; la otra mitad es poder explicar por qué está he-

cho así.

01

El repositorio

Código de los dos servicios, el frontend, el `.c` con su Makefile o script de compilación, y el manifiesto de despliegue. Con un `README` que diga cómo levantar todo desde cero en una máquina limpia.

02

El documento de diseño ADD

Los cinco pasos: drivers, escenarios de calidad, descomposición, interfaces y bitácora. Los escenarios son los tres de este documento más los que agregue el equipo.

03

La bitácora de decisiones

Una fila por decisión, con el formato de abajo. Es lo que más pesa: una decisión sin alternativa evaluada es una decisión que en realidad nunca se tomó.

04

Las pruebas

La suite con los seis conjuntos de control de RQ-02, corriendo en la construcción y con sus resultados visibles.

05

Los cuatro anexos de medición

El `git diff --stat` del quinto algoritmo (RQ-01), la tabla de comparación contra los cálculos de control (RQ-02), las tres sesiones cronometradas con sus preguntas (RQ-03) y el comparativo C contra Java con tamaño de imagen y arranque.

06

La sustentación

El sistema levantado con el manifiesto, delante del docente, y el quinto algoritmo agregado en vivo y cronometrado. El algoritmo se prepara en papel, no en una rama lista para fusionar: el escenario verifica que el diseño lo soporte.

07

La configuración de identidad y la evidencia de acceso

El archivo de importación del realm de Keycloak, versionado en el repositorio, con sus roles, sus clientes y sus usuarios de prueba; y el registro de las cuatro respuestas HTTP —401 sin token, 401 con firma alterada, 403 con rol equivocado en los dos sentidos de la matriz, y 200 con el rol correcto— con el comando ejecutado junto a cada una.

FORMATO DE LA BITÁCORA

DECISIÓN	ALTERNATIVAS DESCARTADAS	JUSTIFICADA CONTRA	QUÉ COSTÓ
El identificador del algoritmo viaja como texto y se valida contra la lista que reporta la biblioteca.	Un `enum` en Java replicado en Angular; un endpoint distinto por algoritmo.	**RQ-01 · Modificabilidad** — el enum obligaba a tocar tres módulos por cada algoritmo nuevo.	Se pierde la validación en tiempo de compilación; hay que devolver un error claro cuando el nombre no existe.

Esa fila es un ejemplo real de este proyecto, y corresponde al nivel de detalle esperado. Ocho o diez filas así valen más que treinta de una línea.

Por dónde empezar

Cinco acciones para la primera semana, suficientes para iniciar el trabajo sin información adicional.

01

Resolver a mano el conjunto de control

En papel, seis procesos, los cuatro algoritmos, las tres métricas. Es el primer artefacto del proyecto y sin él no hay forma de saber si el software está bien. Se fotografía y se sube al repositorio.

02

Escribir el contrato antes que el código

El JSON de entrada y el de salida de `POST /planificaciones`, y el de `GET /algoritmos`. Se guarda como `openapi.yaml` en el repositorio. Debe revisarse contra RQ-01: si el contrato enumera "fifo", "sjf", "rr" y "prioridad" como valores cerrados, se redacta de nuevo.

03

Hacer andar la cadena completa con un solo algoritmo

`libplanificador.c` con FIFO y nada más. Se compila, se prueba desde un `main` en C, después desde Java con JNA, después expuesta por el endpoint, después dibujada en Angular. Solo cuando la cadena entera funcione de punta a punta se agregan los otros tres: en ese punto cada uno toma media hora.

04

Repartir el trabajo por dominio, no por capa, y cruzarlo

Una propuesta: un integrante toma la biblioteca nativa y `planificador-service`; el otro toma `proceso-service` y el frontend. Y una regla que no admite excepción: **nadie puede ser el único que sabe compilar el `.so` o levantar el manifiesto**. Antes de la entrega, cada integrante debe poder hacer andar el sistema completo por su cuenta.

05

Levantar el despliegue vacío en la primera semana

El manifiesto con los dos servicios y el frontend, aunque cada uno solo responda "hola". Dejar la orquestación para el final es el error clásico y el que más proyectos hunde. La bitácora se abre el mismo día, con la primera decisión ya escrita: el criterio del corte entre los dos servicios.

Ampliación opcional: gestión de memoria

Se documenta aquí para dejar constancia de que existe. No debe abordarse antes de que el planificador esté probado.

La continuación natural de este sistema es simular la gestión de memoria: asignar a cada proceso un tamaño, aplicar estrategias de ubicación —primer ajuste, mejor ajuste, peor ajuste— y mostrar la fragmentación resultante junto al Gantt. Encaja con el diseño existente y arquitecturalmente cuesta poco: es otra función en la biblioteca en C y otra vista en Angular, que es justamente lo que RQ-01 deja preparado.

> **La ampliación no debe iniciarse antes de tiempo.** La condición para abrir este frente es que los cuatro algoritmos pasen los seis conjuntos de control, que el sistema se levante con el manifiesto y que la bitácora esté al día. Antes de eso, cada hora dedicada a la memoria se resta de lo que sí se evalúa. Un planificador correcto y bien justificado vale más que un planificador dudoso con un módulo de memoria a medias — y, alcanzada esa condición, el módulo de memoria se vuelve la mejor demostración posible de que la arquitectura aguanta.

El sistema se evalúa contra el alcance definido en este documento. Lo que no se rebaja es la justificación: la bitácora de decisiones pesa tanto como el sistema funcionando, porque un sistema que funciona sin que nadie sepa por qué está hecho así no se puede mantener ni defender. Con el atributo asignado a este proyecto, la justificación *es* el proyecto.